PromoGuard Future Upgrade Plan

A phased product, feature, integration, and release roadmap

Prepared: 26 October 2026

Project baseline: PromoGuard 0.1.0, local Windows Electron prototype

Plan status: Proposed roadmap; future-phase features are not represented as implemented

Purpose
A practical, safety-first sequence for improving campaign evidence, desktop reliability, optional integrations, and future add-ons?without promising earnings or granting financial execution authority.

1. Executive summary

PromoGuard should grow into a **scam-aware crypto opportunity and market-intelligence workstation**. The product should help a person collect public leads, inspect primary-source terms, record evidence, understand uncertainty, and optionally review public wallet data without giving the app authority over funds.

The recommended path is deliberately incremental:

1. Make today's desktop prototype dependable to install, operate, back up, and troubleshoot.
2. Improve evidence capture and campaign review so records are reproducible and honest.
3. Validate each external provider on testnets or with public read-only data before relying on it.
4. Add useful, optional research extensions behind explicit permissions and safe data boundaries.
5. Add live market intelligence only after a provider, license, attribution, and outage strategy are approved.
6. Consider a web companion or sync only as a separate architecture and privacy decision.

This is a **planning document**, not a claim of completion or profitability. A plan cannot promise earnings, eliminate scam risk, certify a campaign, or establish legal approval.

Recommended product boundary

- Campaign discovery and evidence organization are the core product.
- Wallet connections remain optional, read-only, and consent-gated.
- Claim support remains preflight/research only unless a separately reviewed design proves safe receipt attribution.
- Markets may show sourced observations, not investment advice.
- Predictions remain inactive or valueless paper exercises, with no redeemable points, prizes, money, tokens, or payout.
- No exchange, trading, custody, signing, approvals, deposits, transfers, staking, yield, real-money predictions, or financial execution is in scope.

Interpretation of “extensions and add-ons”

This roadmap treats the request as asking for future integrations, modules, and optional add-ons. Each add-on is still subject to evidence, privacy, licensing, security, and user-consent gates; “add all add-ons” is not a safe or useful release requirement.

Aadhaar clarification: Aadhaar is sensitive national identity data, not a normal add-on to a local crypto research tool. This roadmap excludes Aadhaar-number collection, storage, verification, and KYC. If “Aadhaar” meant something else, rename that requirement. If an identity feature is ever genuinely required, first create a separate legal/privacy design, jurisdiction review, data-minimization plan, and explicit approval; do not add Aadhaar data to this application by assumption.

2. Current baseline: what exists and what does not

The current source of truth is the running source tree, tests, and current audit/status reports—not historical completion percentages in older plans or PDFs.

Area	Current baseline	Upgrade implication
Desktop	Windows Electron shell with seven navigable views, isolated renderer, narrow preload API, CSP, local SQLite	Stabilize packaging and operations before adding large features

Area	Current baseline	Upgrade implication
Campaign review	Manual campaign records, status/evidence review, source allow-list, separate source and fee attestations	Add fetched, versioned evidence and clearer provenance; keep unknowns unverified
Discovery	RSS/Atom/GDELT lead parsing, duplicate handling, scheduling consent and request safety limits	Add curated sources, freshness, source-health and source-specific controls
Wallet Center	TON Connect and WalletConnect/EVM source paths, public-data consent, provider/network checks	Needs real testnet acceptance and explicit provider configuration before advertising as live
Claim workflow	Preflight and limited monitoring	Not claim submission; avoid presenting a balance increase as a receipt
AI/MCP	Local advisory AI and per-tool consent	Improve source citations and safe output handling; AI never sets eligibility
Markets	No market provider configured	Choose a licensed, attributed source before building charts
Predictions	Inactive paper/demo placeholder	Keep inactive or implement only valueless, unmistakably simulated exercises
Release	NSIS build works; unsigned; clean-profile installation not certified	Add clean-machine tests, signing, update and support policy
Verification	112 automated tests and mocked integration coverage; smoke at 390/1180px	Add live-provider staging and accessibility/release acceptance

See PROJECT_STATUS.md, PROJECT_AUDIT.md, TEST_REPORT.md, SECURITY_REVIEW.md, and API_AND_DATA_SOURCES.md for the current baseline and its limits.

3. Product outcomes and success measures

Measure product quality and user understanding, not user profit.

Outcomes

1. A user can trace every campaign record to a source and a dated evidence snapshot.
2. Missing, stale, contradictory, or inaccessible evidence is clearly labeled.
3. Users can understand why a campaign is unverified, rejected, expired, or eligible for further review.
4. Discovery sources are manageable, rate-limited, attributable, and removable.
5. Wallet access is optional, read-only, revocable, and transparent about what each provider receives.
6. The application can be installed, upgraded, backed up, restored, and removed with documented data behavior.
7. A release can be reproduced, tested, signed, and supported.

Suggested measurable acceptance targets

- 100% of imported/manual leads start unverified until required evidence gates are met.
- Every campaign review result stores the policy version and references the exact evidence revision used.
- Every optional integration displays provider, purpose, data sent, consent state, and last successful/failed check.
- No renderer-exposed bridge method can sign, approve, transfer, submit a claim, or retrieve a seed phrase.

- No critical/high dependency advisory is left unexplained at release; exceptions require an owner-approved record and mitigation.
- Installer acceptance passes on a clean Windows profile and a supported upgrade profile before public distribution.
- Keyboard-only paths and accessibility checks pass for all release-critical flows.
- Market values display source, timestamp, unit/currency, and stale/unavailable state.

These are engineering quality gates, not guarantees that users will avoid scams or receive rewards.

4. Principles for every future feature

1. **Evidence before conclusions.** A user assertion is not independent verification. A URL is not a fetched snapshot.
2. **Fail closed.** Missing or conflicting facts stay `Unverified`; never map missing data to `false`, “free,” “safe,” or eligible.
3. **Read-only wallet boundary.** No signing, approvals, transfers, private keys, seed phrases, custodial balances, or automated claims.
4. **Consent is specific.** State who receives what data, for what purpose, and how to revoke it.
5. **Source attribution.** Identify provider, capture time, chain, units, confidence/limitations, and any license restrictions.
6. **Graceful degradation.** A provider outage becomes an explicit error/stale state, never fabricated or success-shaped data.
7. **Minimal data.** Store only what supports the user's local research task. Sensitive identity data is excluded.
8. **Safe extension model.** Extensions are curated and permission-scoped; do not load arbitrary remote executable code.
9. **Migrations preserve data.** SQLite migrations are append-only and tested against representative older databases.
10. **Release gates are real gates.** A passing unit suite is not a live service test, penetration test, legal review, or clean-machine installation test.

5. Roadmap at a glance

Effort is expressed as rough engineering person-week ranges for planning only. Actual estimates depend on the maintainer team, provider selection, design decisions, review capacity, and acceptance findings. Phases should be gated by demonstrated evidence, not dates alone.

Phase	Name	Approx. effort	Priority	Exit gate
0	Stabilize and release foundation	1–2 person-weeks	P0	Clean Windows install/upgrade/uninstall and documented data behavior pass
1	Evidence-grade campaign workspace	2–4 person-weeks	P0	Snapshot/hash, provenance, version comparison and fail-closed policy tests pass
2	Discovery source management	2–3 person-weeks	P1	Curated feed workflow, freshness/error visibility and abuse limits pass
3	Wallet integration acceptance	2–4 person-weeks plus owner setup	P1	Real testnet consent/revocation/erase flows pass; no signing surface exists

Phase	Name	Approx. effort	Priority	Exit gate
4	Research extensions/add-on framework	3–5 person-weeks	P1	Curated extension registry and least-privilege permission tests pass
5	Market intelligence	3–6 person-weeks plus provider contract	P2	Licensed source, attribution, staleness and outage tests pass
6	Valueless paper research lab	2–4 person-weeks	P2	Simulated content is unmistakably fictional, isolated, non-redeemable, and tested
7	Optional backup/export and platform expansion	3–8 person-weeks	P3	Restore, encryption, schema versioning, privacy and threat review pass

Recommended first delivery: Phase 0, then Phase 1. Avoid starting all phases concurrently.

6. Phase 0 — Stabilize the release foundation

Scope

- Establish supported Windows versions, Node/Electron versions, install paths and storage paths.
- Test the generated NSIS installer on a clean local Windows profile.
- Test first launch, application restart, data persistence, upgrade, uninstall, and reinstall.
- Decide whether uninstall preserves local research data by default and how a user can explicitly erase it.
- Add app metadata, a locally supplied icon, publisher name, version policy, and code-signing plan.
- Define a reproducible release procedure: locked dependencies, test commands, build output, checksums, release notes and rollback.
- Keep sensitive session/configuration data in OS-backed encryption; explain behavior when safe storage is unavailable.
- Retain the narrow EPIPE-only stream handling; do not add a broad uncaught-exception suppression.

Example acceptance run

1. Create a clean Windows test account or VM snapshot.
2. Install `promoguard Setup <version>.exe`.
3. Launch from Start Menu; create a fictional campaign record; close and reopen.
4. Verify the record remains and the migration version is current.
5. Install a higher test build over the existing install; verify data remains readable.
6. Uninstall; verify the documented retention behavior.
7. Reinstall; verify the app handles retained or absent user data safely.
8. Record screenshots, OS/version, artifact hash, pass/fail, and any crash logs.

Exit criteria

- No unhandled startup errors in clean profile.

- Installer and app are publisher-attributed and signed before public release (or explicitly internal-only).
- Data retention/deletion is understandable and tested.
- Crash reporting does not upload campaign text, addresses, or secrets without explicit opt-in.

7. Phase 1 — Evidence-grade campaign workspace

Features

1. **Evidence capture**
2. Add a user-triggered “Fetch official terms” action.
3. Fetch only public HTTPS pages with strict DNS/IP validation, redirect limits, byte caps, content-type checks, timeouts, and host throttling.
4. Do not execute page scripts or render remote HTML as trusted application markup.
5. Store normalized text or a safe document snapshot, canonical URL, fetch time, response metadata, content hash, and fetch errors.
6. Preserve prior revisions; never silently overwrite evidence.
7. **Evidence timeline**
8. Show what changed between revisions.
9. Mark removed, changed, stale, failed, and user-entered evidence distinctly.
10. Keep raw source and derived summary separate.
11. **Review explanation**
12. Show deterministic policy version, each requirement, evidence reference, conclusion, uncertainty, and next action.
13. Make “eligible for review” visually distinct from independent verification or payout.
14. **Campaign lifecycle**
15. Add safe edit, archive, restore, and delete-with-confirmation workflows.
16. Keep an audit/history trail for decisions and retain tombstones or backup protections as appropriate.
17. **User export**
18. Export a campaign evidence bundle in a documented JSON/ZIP format.
19. Include source URLs, hashes, timestamps, policy version, and provenance labels.
20. Warn that a hash proves file identity, not truthfulness of the source.

Example: official terms revision changes

```
Campaign: Example community reward
Revision A fetched: 2026-10-26 10:00 UTC
Revision B fetched: 2026-10-28 10:00 UTC
Change detected: “No purchase required” section changed
Result: Previous review invalidated; status returns to UNDER_REVIEW
Action: User must re-read the current official terms
```

This fictional example demonstrates a workflow only; it is not an actual campaign.

Exit criteria

- Tests prove each evidence revision is immutable and correctly hashed.
- Any failed fetch remains an explicit failure, not success-shaped content.
- A changed terms hash invalidates prior review.
- Scripted content cannot execute in the application.
- Policy evaluation is deterministic, test-covered, and versioned.

8. Phase 2 — Discovery source management

Features

- Curated, categorized source catalog with source owner, purpose, format, last success, last error and retention details.
- Add/remove/import feed configuration with preview before saving.
- Per-source polling interval, concurrency and rate limits; user consent for scheduled polling.
- Per-host backoff and circuit breaker for repeated failures.
- Lead confidence is a transparent metadata score (e.g. source freshness/format quality), never a legitimacy verdict.
- Deduplication across URL variants and provider IDs, while preserving source attribution.
- Review queue with filters: new, duplicate, source changed, stale, rejected, needs manual review.
- Optional export of lead list; do not auto-contact projects or post on social media.
- No credential scraping or private/group access.

Example

```
Source: Example project announcement feed (fictional)
Last checked: 2026-10-26 12:00 UTC
Result: 3 entries parsed, 1 new lead, 1 duplicate, 1 skipped
Lead status: UNVERIFIED
Reason: No fetched official terms evidence yet
```

Exit criteria

- A malicious redirect to a private/local address is rejected.
- Maximum bytes, timeout, redirect count and request rate are enforced.
- A user can disable a source immediately and see its last error.
- Leads never become eligible solely because a feed is trusted or scheduled.

9. Phase 3 — Live wallet integration acceptance (read-only)

This phase validates the existing wallet integration paths rather than expanding their authority.

TON testnet

- Owner hosts the TON Connect manifest on public HTTPS and verifies its content and domain.
- Pair a dedicated test wallet on TON testnet.

- Exercise connect, disconnect, restore, wrong-network behavior, missing manifest, wallet rejection, provider outage, rate limit, consent grant/revoke, local wipe, and reconnect.
- Verify no signing/send API or private data is stored.

EVM/WalletConnect testnet

- Owner creates a Reown project ID and reviews its terms/allowed origins.
- Configure a verified HTTPS testnet RPC endpoint.
- Pair a dedicated test wallet; confirm only the expected chain/account is accepted.
- Test RPC chain mismatch, provider timeout/429, disconnect, session restore, lookup consent/revocation, and local erasure.
- Inspect requested namespaces/methods; request no signing/transaction methods.

Example consent disclosure

> *Allow balance lookup for this public address on Example Testnet? The selected RPC provider receives the address, chain request, and network metadata such as your IP address. No wallet keys are sent. You can revoke this provider's lookup permission at any time.*

Exit criteria

- Real testnet behavior is documented by date, provider, network and app version.
- No chain mismatch returns a balance.
- Revocation prevents subsequent lookup requests.
- Erasure removes scoped local cache and permission records.
- Live results are never represented as campaign payment proof without a verifiable campaign-specific receipt.

10. Phase 4 — Safe extensions and add-ons

“Extensions” should mean curated product capabilities, not arbitrary plugin code with access to the local database or wallet session.

Proposed add-on catalog

Add-on	Purpose	Required permission	Initial recommendation
Evidence Snapshot	Capture and compare public campaign terms	User-triggered HTTPS fetch	Build in Phase 1
Feed Packs	Install reviewed public feed lists	Read listed public feeds after user approval	Curated first-party packs only
Risk Checklist Pack	Add transparent, deterministic review questions	Read campaign text provided by user	Add after policy review
Export Bundle	Export a user's local campaign/evidence records	User-triggered local file write	Build with Phase 1
Market Data Connector	Display sourced prices/market metadata	Provider network access and attribution	Wait for provider/license selection
Explorer Link Pack	Open recognized public explorer pages	External link only	Allow-list domains and require click
Local AI Provider	Summarize user-selected evidence locally	Explicit local model/configuration	Existing direction; cite source passages
MCP Tool Pack	Read-only local campaign analysis	Explicit per-tool capability grant	Extend only after threat review
Browser/Office import	Import user-selected documents or URLs	User-selected file/network read	Defer until malware/document parser review
Identity/KYC integration	Identity proofing	Highly sensitive data and jurisdiction-specific controls	Excluded; separate product/legal decision

Extension architecture

- First release supports only first-party reviewed packs. No arbitrary JavaScript execution, `eval`, dynamic remote module loading, or native code plugins.
- Extensions declare a static manifest: identifier, version, publisher, purpose, requested permissions, network hosts, data read/write scope, retention, and update source.
- Permission sets are capability-based (e.g. `campaign.read`, `evidence.fetch`, `market.read`, `export.write`), not broad filesystem/database access.
- User approves each capability before activation and can revoke at any time.
- Any extension update that adds permissions requires renewed consent.
- Extension output is untrusted data, sanitized before display, and cannot change deterministic policy decisions.
- The host provides versioned APIs. Extensions cannot access wallet session keys, safeStorage keys, private app internals, or arbitrary SQL.
- Sign extension packages, pin hashes, maintain a reviewed allow-list, publish a security contact, and support emergency disable.
- Provide an audit view showing extension version, permissions, network hosts contacted, and last use.

Example add-on install flow

1. User selects “Install Official Feed Pack.”
2. PromoGuard shows publisher, feeds/domains, refresh interval, and exact requested capabilities.
3. User previews feeds and chooses whether to enable manual or scheduled checks.
4. App stores the pack manifest/version and consent state.

5. Each lead still starts unverified; the pack cannot approve or mark a campaign eligible.
6. User disables the pack; scheduled requests stop immediately and local data retention is explained.

Exit criteria

- Permission escalation prompts for renewed user approval.
- No extension can invoke signing or transaction calls because those APIs do not exist.
- Extension content cannot execute as code or bypass URL/security validation.
- Extension disable/revoke is tested and reflected in logs and UI.

11. Phase 5 — Market intelligence

Decisions required before coding

- Which asset classes are in scope: crypto only, or also equities/commodities?
- Which jurisdictions and user groups are served?
- Which market data provider grants display and redistribution rights?
- Which units, currencies, attribution and delayed/live labeling are required?
- What happens when provider data is stale, missing or disagrees?

Do not scrape provider pages or ship a “live” chart from sample values.

Features after provider approval

- Watchlists and asset details with provider attribution, timestamp, currency and freshness.
- Clearly labeled delayed versus real-time data.
- Range/time controls with accessible non-color-only legends.
- Source status panel and rate-limit/error explanation.
- Optional price alert rules evaluated locally with explicit notification consent.
- No buy/sell signals, price predictions, guaranteed returns, or automated orders.
- No mixing of market-data values with campaign evidence or reward balances.

Example data card

```
Asset: Example Token (fictional)
Price: unavailable
Source: Example provider (not configured)
Last successful update: none
Status: configure a licensed data source before showing a quote
```

Exit criteria

- Provider license and attribution have owner approval.
- Every value carries source/time/unit and can be identified as stale.
- Provider failures never repeat cached values as current.
- Automated checks cover malformed, delayed, outlier, and unavailable responses.

12. Phase 6 — Paper research lab (optional)

Paper mode should teach/organize research, not simulate a guaranteed strategy.

Allowed design

- Explicit “SIMULATION — NO MONEY OR VALUE” banner on every view and export.
- Hypothetical scenarios created by the user or clearly marked fixtures.
- No deposits, transfers, redeemable points, prizes, tokens, or payouts.
- No fake “live” odds or outcomes. Historical data must cite its source and date.
- No leaderboard that rewards risky behavior or implies user earnings.
- Reset/delete simulated records separately from real campaign records.

Example paper exercise

```
Scenario: Example Token may move above a user-entered threshold
Mode: Simulation only — no money, points, prize or payout
Observation source: Not connected
Outcome: Not evaluated
```

If a user later requests real-money prediction/betting or trading, stop feature implementation and require a separate legal/compliance, licensing, age/location, AML/KYC, custody/payment, market-integrity, independent-security and disclosures review. A disclaimer is not enough.

13. Phase 7 — Backup, export, and optional platform expansion

Local backup and restore

- User-initiated backup with schema version, creation time, integrity checksum, and clear included/excluded data inventory.
- Option to exclude wallet connection/session configuration and all provider credentials.
- Encrypt backup with a user-selected passphrase only if a professionally reviewed format and recovery procedure are selected.
- Verify imported archive size/type and validate before changing live records.
- Restore into a preview/staging view; show conflicts and counts before applying.
- Preserve an automatic pre-restore backup; require explicit confirmation.
- Never silently merge duplicate campaign IDs or overwrite newer evidence revisions.

Optional web companion

Only after desktop reliability and security foundations are complete:

- Define separate threat models for local desktop and hosted service.
- Start with read-only campaign/evidence access, not remote wallet control.
- User authentication, tenancy, encryption, backup, deletion, incident response and data residency need a separate architecture review.

- Never sync wallet session secrets or seed/private-key material.
- Provide offline behavior and clear sync conflict resolution.
- Do not add cloud storage just to deliver the current desktop features.

Extension marketplace

Consider only after the first-party extension API and audit controls have matured:

- Publisher verification and package signing.
- Malware/supply-chain review and repeatable builds.
- Permission diff and update rollback.
- Abuse reporting, emergency revocation, vulnerability contact, and support policy.
- Do not permit user uploads of arbitrary executable add-ons in the initial marketplace.

14. Technical architecture and migration plan

Keep the current security boundary

```
Renderer UI
    ■ typed/allow-listed preload calls only
    ▼
Electron main process
    ■■■ campaign policy and source validation
    ■■■ database repositories and append-only migrations
    ■■■ constrained provider adapters
    ■■■ safeStorage-backed session/config encryption
    ■■■ audit and local notification service
```

- Renderer cannot access Node.js, SQLite, secrets, or arbitrary network APIs.
- Add new IPC channels only for concrete use cases; validate sender and every input.
- Avoid returning raw provider payloads or credentials to the UI.
- Use repository/service modules for business logic rather than putting policy into click handlers.
- Maintain backwards-compatible, append-only migrations and test a copied older database fixture.
- Pin extension/provider API schemas; validate JSON responses and impose response limits.
- Keep all external-link opening behind URL validation and user click.
- Do not expose a generic “run command,” “execute SQL,” arbitrary file path or plugin code IPC.

Data model additions to consider

Add only after the relevant phase design. Possible tables/entities:

- `evidence_revisions`: campaign ID, source URL, fetched time, content hash, normalized text reference, fetch status, policy version.
- `evidence_changes`: previous/current revision, diff metadata, invalidation reason.
- `source_registry`: source URL/host, kind, user label, enabled state, interval, consent and last result.
- `provider_health`: provider ID, last success/failure, freshness, retry-after; do not store secrets.
- `extension_registry`: extension ID/version/hash/publisher, requested and granted permissions, enabled state.

- `export_jobs`: operation metadata and result; avoid logging full campaign contents.
- `audit_events`: minimal security-relevant event, timestamp and opaque reference; retention defined.

Every schema addition needs a migration, rollback/read-compatibility strategy, fixture database test, and a data-retention decision. Never store Aadhaar or other national identity numbers as an incidental feature.

15. Testing strategy and release gates

Automated checks

- Unit tests for policy decisions, date/time boundaries, URL parsing, evidence hashes, extension permission evaluation, stale data and error mapping.
- Database migration tests from representative legacy versions with existing user rows.
- Integration tests with mocked providers for timeout, invalid JSON, 429, 5xx, malformed content, chain mismatch and cancellation.
- Renderer tests for loading, empty, stale, failure, consent, revocation, keyboard and screen-reader states.
- Electron smoke tests at narrow and desktop widths; assert renderer isolation, CSP, IPC allow-list and absence of signing/send functions.
- Dependency audit, license inventory, lockfile integrity, secret scan and packaging reproducibility checks.
- Accessibility checks: keyboard-only completion, focus order, visible focus, labels, contrast, zoom and reduced motion.

Manual staging acceptance

- Real TON/EVM testnet pairing using dedicated test wallets only.
- Live provider privacy/terms and rate-limit review.
- Clean Windows install/upgrade/uninstall and data-retention test.
- Backup/restore drill using non-sensitive test records.
- Accessibility review with assistive technology.
- Independent security review before broad distribution.

Go/no-go criteria

Do not release if any of the following is true:

- Critical/high security finding is unmitigated.
- Campaign can become “eligible” with unknown required terms.
- Provider data can be mislabeled live/current when stale or unavailable.
- Consent cannot be revoked or a provider call continues after revocation.
- An extension can run arbitrary code or access wallet session material.
- Installer/data migration loses or corrupts user data.
- Release claims imply guaranteed earnings, legitimacy, legal approval or proven payment.

16. Privacy, legal, and operational readiness

Before any public release, the owner should obtain appropriate professional advice and make explicit decisions about:

- Intended jurisdictions and audience.
- Data controller/operator identity and contact information.
- Privacy notice and retention/deletion policy.
- Provider terms, attribution rights, rate limits and commercial-use permissions.
- Consumer-protection and financial-regulation boundaries.
- Accessibility and support obligations.
- Vulnerability disclosure and incident response.
- Code-signing certificate ownership, update channel, rollback and compromised-release plan.
- Whether any analytics are needed; default to no tracking and collect only with clear opt-in.
- Backup encryption, key loss, recovery, and local data export.

This document is not legal advice. It does not establish regulatory compliance.

17. Risk register

Risk	Impact	Mitigation	Gate
Fake or compromised campaign source	Loss or harm to users	Source provenance, evidence snapshots, independent source checks, explicit uncertainty	Phase 1
Terms change after a review	Wrong conclusion from stale evidence	Revision hashes, freshness labels, review invalidation	Phase 1
Feed/URL SSRF or oversized content	Local network exposure/resource exhaustion	Safe URL resolution, redirect validation, byte/time limits, rate limits	Phases 1–2
RPC/provider lies, fails, or leaks metadata	Incorrect balance or privacy exposure	Explicit consent, chain verification, provider disclosure, stale/error state	Phase 3
Extension supply-chain compromise	Code execution/data exposure	No arbitrary code, signed curated manifests, least privilege, emergency disable	Phase 4
Market-data licensing or attribution breach	Takedown/legal/business risk	Select provider and approve license before implementation	Phase 5
AI hallucination or prompt injection	Misleading user or unsafe action	AI advisory only, source citations, untrusted-input treatment, deterministic policy	All phases
Upgrade migration/data loss	User records lost	Append-only migrations, backups, fixture tests, clean-profile upgrade gate	Phase 0+
Sensitive identity data added without need	Privacy/legal exposure	Exclude Aadhaar/KYC unless separate approved product requirement	All phases
Unsigned or compromised distribution	Malware trust and update risk	Signing, reproducible build, checksums, release review and incident plan	Phase 0

18. Owner decision checklist

Before starting the next engineering cycle, the product owner should decide:

1. **Priority:** Is the next release a private prototype or a public release candidate?
2. **Evidence:** Approve Phase 1 evidence snapshots as the first feature expansion?
3. **Providers:** Which named TON/EVM testnets and providers may be used for live acceptance?
4. **Markets:** Is crypto-only market data sufficient, and which provider/license is acceptable?
5. **Add-ons:** Should extensions remain first-party curated for the first release? Recommended: yes.
6. **Aadhaar:** Confirm excluded unless a separate, justified identity feature is requested. Recommended: exclude.
7. **Data:** Should uninstall preserve campaign records by default, with explicit erase controls? Decide and test.
8. **Distribution:** Who owns the publisher identity, code-signing certificate, support address and release channel?
9. **Budget:** Assign owner/time for independent security, accessibility, and legal reviews.
10. **Scope freeze:** Keep trading, transactions, claims, real-money predictions, and earnings promises out of scope.

19. Suggested first 30-day sequence

This is a sequence, not a promise of calendar completion.

Week 1: Release foundation

- Create a clean Windows test profile/VM.
- Install, launch, restart, upgrade and uninstall the current build.
- Record data retention and safeStorage behavior.
- Resolve installation bugs before adding features.

Week 2: Evidence design

- Approve evidence revision schema and fetch threat model.
- Prototype a user-triggered terms snapshot with strict fetch limits.
- Add tests for redirect-to-private, oversized page, changed hash, malformed charset, timeout and duplicate revisions.

Week 3: Review UX

- Show source, capture time, evidence revision, policy rule outcomes and uncertainty.
- Test accessible keyboard flow and responsive layouts.
- Ensure an evidence change invalidates a prior review.

Week 4: Testnet and release gate

- Owner prepares manifest, Reown project ID and dedicated test wallets.
- Run controlled testnet pairing, revocation and erasure checklist.
- Run full tests, audits, packaging and smoke.
- Hold release if the clean-install, wallet-consent, or evidence gates fail.

20. Example end-to-end future workflow

1. **Discover:** a curated public feed surfaces a fictional “Example community reward” lead.
2. **Triage:** PromoGuard labels it `UNVERIFIED`, stores the feed identity and first-seen time.
3. **Source check:** user navigates to the independently checked official domain; PromoGuard does not auto-approve the domain.
4. **Evidence capture:** user requests a safe snapshot of terms. The app records URL, fetch time, content hash and warnings.
5. **Review:** deterministic checks list each payment condition as confirmed absent, present, or unknown. Any unknown blocks eligibility.
6. **Change detection:** a later snapshot differs, so the previous review is invalidated.
7. **Optional wallet view:** user separately consents to a public address lookup on a testnet provider; app shows provider and freshness.
8. **Closeout:** user archives or exports the evidence bundle. A later balance rise is not labeled as a reward without campaign-specific receipt evidence.
9. **Outcome:** third-party project decisions and reward delivery remain outside PromoGuard's guarantees.

All names and offers above are fictional examples.

21. Final recommendation

Build in this order:

clean installer/data lifecycle → evidence snapshots and review invalidation → discovery operations → live testnet wallet acceptance → curated permissioned add-ons → licensed market data → optional valueless paper research → separately reviewed backup/web expansion.

Do not attempt to make every aspiration “real” in one release. Deliver one phase, prove it with tests and user acceptance, update the status report, then proceed. This is the safest path to a professional and maintainable product while preserving user trust.